
UNIVERSITÉ HASSAN 1^{er}
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES (ENSA)
BERRECHID

Cycle Ingénieur : Big Data & Systèmes d'Information

**RAPPORT DE PROJET DE FIN DE
MODULE**

Application de Gestion des Ressources Humaines
(GRH)

Module : Bases de données avancées

Technologies : JavaFX, Scene Builder, Hibernate 4.3, MySQL 8.0

IDE : NetBeans 8.2

Réalisée par :

Meriem ER.RAHMOUNI

Encadrée par :

Mr. Hamid HRIMECH

Remerciements

Je tiens à exprimer ma profonde gratitude à mon professeur et encadrant de ce module de Bases de Données Avancées, pour son accompagnement précieux et ses conseils avisés tout au long de ce projet. Ses enseignements théoriques et pratiques ont été essentiels pour mener à bien la conception de cette application de gestion des ressources humaines, notamment pour l'intégration de technologies exigeantes telles que JavaFX, Hibernate et MySQL. Sa rigueur scientifique et sa disponibilité constante m'ont non seulement permis de surmonter les défis techniques liés à la persistance des données, mais ont également grandement contribué à enrichir mes compétences en ingénierie logicielle.

1 Introduction Générale

Dans un environnement entrepreneurial de plus en plus compétitif et numérisé, la gestion efficace du capital humain est devenue un pilier stratégique pour toute organisation. La Gestion des Ressources Humaines (GRH) englobe un ensemble de processus complexes allant du suivi administratif des collaborateurs à la gestion structurelle de l'entreprise.

L'objectif de ce projet est de concevoir et de développer une application de bureau moderne et performante dédiée à la GRH. Cette solution informatique se concentre sur trois fonctionnalités centrales : l'organisation de l'entreprise en **départements**, le suivi rigoureux des **profils des employés**, et la gestion dynamique de leurs **absences**.

Sur le plan technique, ce projet met en pratique les concepts clés du génie logiciel étudiés en cycle d'ingénieur. Pour offrir une expérience utilisateur fluide, l'interface graphique a été conçue sous l'environnement **JavaFX**. Afin d'éviter les inconvénients du SQL natif, la couche d'accès aux données s'appuie sur le framework de persistance **Hibernate (ORM)**, assurant une communication transparente et orientée objet avec la base de données relationnelle **MySQL**.

2 Technologies Utilisées

Technologie	Version	Rôle dans le projet
JavaFX	8.0.202	Construction de l'interface graphique (fenêtres, tableaux, formulaires)
Scene Builder	2.0	Conception visuelle des fichiers .fxml par glisser-déposer
Hibernate ORM	4.3.1	Couche de persistance : mapping objet-relationnel et transactions BDD
MySQL	8.0	Système de gestion de base de données relationnelle
Java JDK	1.8.0_202	Langage de programmation principal
NetBeans IDE	8.2	Environnement de développement intégré

TABLE 1 – Stack technologique du projet GRH

3 Architecture Logicielle et Démarche de Réalisation

3.1 L'Architecture MVC (Modèle-Vue-Contrôleur)

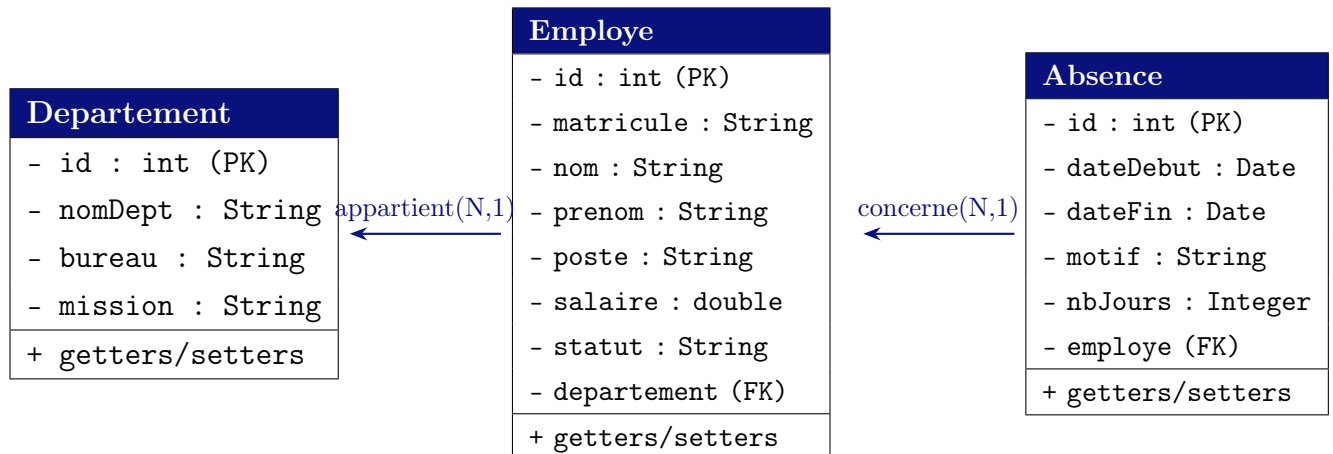
L'organisation de l'application repose sur le patron d'architecture **MVC**, séparant rigoureusement les responsabilités :

- **Le Modèle** : Représenté par les entités métiers JavaBeans (`Employe`, `Absence`, `Departement`) situées dans le package `classes.beans`. Chaque classe est annotée avec les annotations JPA/Hibernate (`@Entity`, `@Table`, `@Column`, `@ManyToOne`).
- **La Vue** : Conçue visuellement sous Scene Builder. Chaque interface est décrite par un fichier `.fxml` indépendant : `FXMLDocument.fxml` (fenêtre principale avec onglets), `FormulaireEmploye.fxml` et `FormulaireAbsence.fxml` (pop-ups).
- **Le Contrôleur** : Représenté par des fichiers Java dédiés à chaque vue (`FormulaireEmployeController`, `FormulaireAbsenceController`, `FXMLDocumentController`). Cette séparation évite d'avoir un contrôleur principal surchargé.

4 Modélisation UML et Structure Relationnelle

4.1 Diagramme de Classes

Le cœur du système repose sur trois entités interconnectées :



4.2 Schéma de la Base de Données gestion_rh

Table	Colonne	Type	Contrainte
departement	id_dept	INT	PK, AUTO_INCREMENT
	nom_dept	VARCHAR(100)	NOT NULL
	bureau	VARCHAR(50)	
	mission	TEXT	
employe	id_emp	INT	PK, AUTO_INCREMENT
	matricule	VARCHAR(20)	UNIQUE
	nom	VARCHAR(50)	NOT NULL
	prenom	VARCHAR(50)	NOT NULL
	poste	VARCHAR(100)	
	salaire	DOUBLE	
	statut	VARCHAR(30)	
	id_dept	INT	FK → departement
absence	id_abs	INT	PK, AUTO_INCREMENT
	date_debut	DATE	NOT NULL
	date_fin	DATE	NOT NULL
	motif	VARCHAR(200)	
	nb_jours	INT	
	id_emp	INT	FK → employe

TABLE 2 – Structure relationnelle de la base de données gestion_rh

5 Spécifications Fonctionnelles Réalisées

L'application offre un espace de gestion interactif divisé en trois modules accessibles via des onglets :

5.1 Module Gestion des Départements

Ce module permet de gérer la structure organisationnelle de l'entreprise. L'utilisateur peut saisir le nom, le bureau et la mission d'un département via un formulaire intégré directement dans l'onglet principal. Les opérations disponibles sont :

- **Ajouter** un département : la ligne apparaît immédiatement dans la table et est persistée en base via `session.save()`.
- **Supprimer** un département sélectionné : suppression en BDD via `session.delete()` et mise à jour de la table.
- **Annuler** : réinitialisation des champs de saisie.

5.2 Module Gestion des Employés

Ce module permet le suivi complet des collaborateurs. Un clic sur "*Ajouter un employé*" ouvre une fenêtre pop-up modale contenant :

- Des champs de saisie pour le matricule, nom, prénom, poste, salaire et statut.
- Un `ComboBox` chargé dynamiquement depuis la BDD affichant tous les départements disponibles.
- Un bouton **Enregistrer** qui déclenche la persistance Hibernate et ferme le formulaire, puis rafraîchit automatiquement la table principale.

5.3 Module Gestion des Absences

Ce module gère les congés et absences des employés. L'interface propose :

- Un champ matricule pour identifier l'employé (recherche HQL par matricule).
- Deux `DatePicker` garantissant la validité du format des dates.
- Un champ motif et le calcul automatique du nombre de jours d'absence.
- Persistance en BDD et rafraîchissement en temps réel de la table.

5.4 Mise à Jour en Temps Réel

L'intégration d'`ObservableList` couplée aux requêtes HQL (`from Employe`, `from Absence`, `from Departement`) permet de rafraîchir les `TableView` dès la fermeture de chaque formulaire d'ajout, sans redémarrer l'application.

6 Difficultés Rencontrées et Solutions Apportées

Le développement de ce projet a nécessité la résolution de plusieurs problèmes, dont les principaux sont détaillés ci-dessous :

6.1 Incompatibilité de la syntaxe Hibernate

La méthode `createQuery()` de Hibernate 5+ accepte un second paramètre de type `Class<T>` pour le typage fort des résultats. Or, le projet utilise **Hibernate 4.3.1**, qui ne supporte pas cette syntaxe :

Listing 1 – Syntaxe incompatible (Hibernate 5)

```
1 // Provoque une erreur de compilation sous Hibernate 4
2 session.createQuery("from Employee", Employee.class).list();
```

Listing 2 – Syntaxe correcte (Hibernate 4)

```
1 // Solution adoptée : cast explicite
2 List<Employee> liste = session.createQuery("from Employee").list()
   ;
```

6.2 Valeur nulle sur un type primitif (nbJours)

Hibernate levait une exception lors du chargement des absences car le champ `nbJours` était déclaré comme type primitif `int`, incompatible avec les valeurs `NULL` en base de données :

Listing 3 – Correction du type nbJours

```
1 // Avant : type primitif, incompatible avec NULL en BDD
2 private int nbJours;
3
4 // Après : type objet Integer, accepte NULL
5 @Column(name = "nb_jours")
6 private Integer nbJours;
```

7 Conclusion Générale

Ce projet de fin de module a été une opportunité majeure pour assimiler et mettre en œuvre de manière concrète l’architecture des applications d’entreprise orientées objet.

L’apprentissage s’est avéré particulièrement riche dans la restructuration d’une interface graphique modulaire avec JavaFX, forçant un découpage propre du code en plusieurs contrôleurs indépendants. De plus, la manipulation du framework Hibernate a démontré l’immense valeur ajoutée des technologies ORM, permettant de s’affranchir de la syntaxe SQL répétitive pour se concentrer pleinement sur la logique métier.

Ce travail pose des jalons indispensables pour aborder sereinement le développement d’architectures logicielles plus complexes et scalables, notamment dans le cadre de projets Big Data et Systèmes d’Information.